

Hierarchical Temporal Memory (HTM)

Hierarchical Temporal Memory (HTM) is a theoretical framework inspired by the structural and functional characteristics of the human neocortex. It acts to emulate biological processes of learning through continuous prediction of temporal sequences, adaptation to new information and anomaly detection. The primary objectives of HTM are to emulate how the human brain processes information and to develop machine learning systems capable of learning in a manner similar to human cognition.

Key concepts in HTM encompass:

Spatial Pooling: Responsible for learning a sparse representation of the input space which enables efficient representation of unique structures and the co-occurrence of input features.

Temporal Memory: Focuses on learning temporal sequences and predicting elements based on previously observed data.

Sparse Distributed Representation (SDR): In contrast to dense matrices used in most neural networks HTM utilizes sparse representations where only a small fraction of neurons remains active this approach improves efficiency and storage capacity.

Background of HTM

HTM networks are based from the work of Jeff Hawkins and the Numenta research team who wanted to replicate how the human neocortex understands and processes information. HTM is used to solve challenges such as sequence learning, pattern recognition, anomaly detection, and prediction through the application of core neuroscience principles. In contrast to other neural networks, HTM uses sequence memory, local learning, and sparse representations, aligning closely with the biological functioning of neural systems.

Mechanisms of HTM

The learning process within HTM is characterized by two primary components:

Spatial Pooler (SP): Designed to derive stable representations for recurring spatial patterns by converting input into sparse representations. It ensures that common features activate similar columns of neurons, thereby establishing consistent representations. By Using memristor-based implementations of the spatial pooler, which significantly improve efficiency by leveraging the properties of memristors, such as low power dissipation and nanoscale integration. These implementations allow for on-device learning and parallel training, thereby reducing reliance on CMOS circuits and improving scalability.

Temporal Memory (TM): Identifies sequences by creating connections between neurons based on temporal context, predicting subsequent occurrences based on learned past sequences. Utilizing spin-neurons and resistive memory has further enhanced the energy efficiency of temporal memory implementations. Spin-neurons combined with resistive

crossbars have demonstrated over 200× lower energy dissipation compared CMOS-based designs, making them particularly suitable for low-power applications.

Mathematically, HTM employs sparse distributed representations for encoding inputs and learning sequences, facilitating efficient information storage. Unlike neural networks, HTM does not require backpropagation; instead, it employs local, Hebbian-like learning rules, consistent with biological principles.

Constructing an HTM Neural Network in MATLAB Simulink

An HTM model can be constructed in MATLAB Simulink by forming a network of interconnected blocks that represent neurons, synapses, and processing units. The Simulink diagram provided suggests that multiple interconnected elements represent core HTM operations such as learning and inference. The KWTa (k-Winners-Take-All) block within the Simulink model is responsible for enforcing sparsity by selecting the most active columns among the neurons.

Simulink KWTa Block Overview

The KWTa block in the Simulink diagram ensures competitive activation among columns, thereby enforcing the desired sparsity. The KWTa mechanism operates by selecting a limited number of winning neurons, ensuring that only those with the highest activations are retained while the others are inhibited.

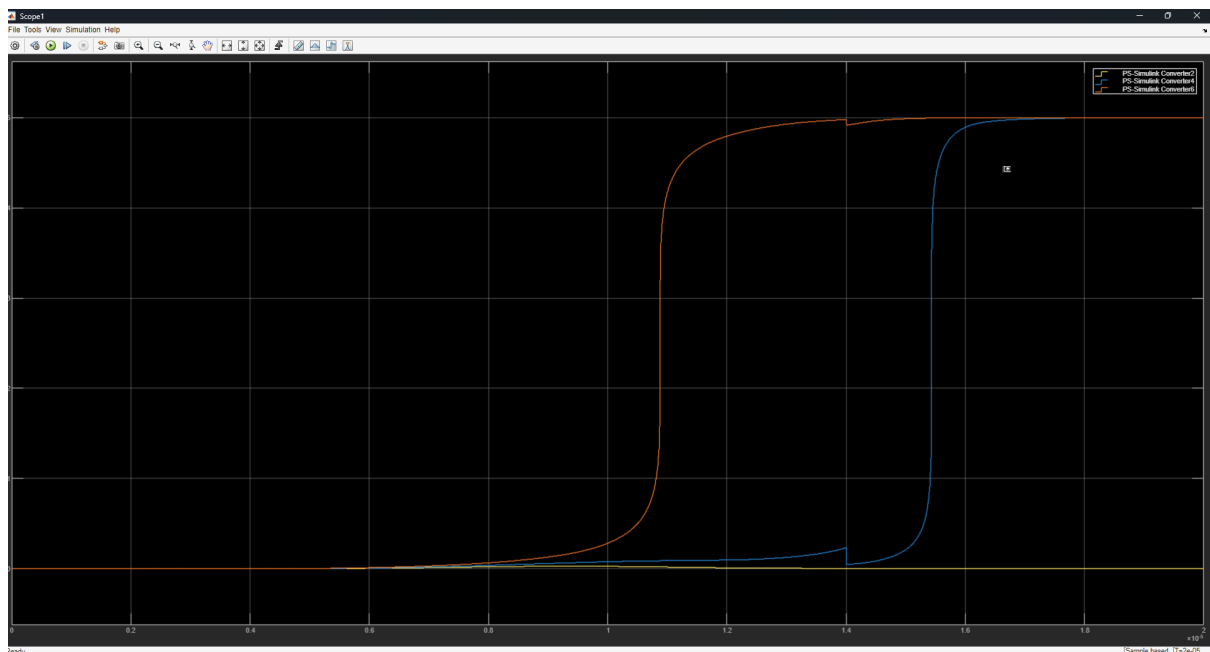
Input Signals: The KWTa block receives multiple input signals, which represent the activation strengths of columns based on their overlap scores or input features. These signals serve as the basis for determining which columns are to be activated.

Processing Elements: The block processes these input signals by performing operations like normalization, comparison, and competitive inhibition. This helps in identifying which subset of columns exhibits the strongest responses aligning with the principles of sparse distributed representations.

Outputs: The outputs of the KWTa block indicate which columns are active after the competition process, ensuring that only a fraction of neurons within a column achieve the winning status. This competitive selection used in the sparsity of HTM's representation.

Graph Analysis and Interpretation

Graph 1 (Scope 1):



The graph demonstrates the activation dynamics for multiple neurons or columns using the k-Winners-Take-All (kWTA) mechanism, which is fundamental in understanding feature detection and selection within Hierarchical Temporal Memory (HTM) networks. In this context, each curve corresponds to a competitive activation of different columns when a specific input threshold is surpassed.

Competitive Activation Dynamics

The sudden and steep rise observed in each curve corresponds to a critical point where the input strength has crossed the activation threshold, which results in the competitive selection of winning neurons or columns. This sudden activation signifies that particular neurons or columns are now actively representing the input. In the given graph, the activation threshold appears to be crossed between approximately 0.8 to 1.0 (time units), where each curve begins to increase rapidly.

The behavior captured here effectively demonstrates how specific columns win the competition based on their input strengths, which is critical for distinguishing which features of the input are most relevant. For instance, the orange line crosses the threshold before the blue line, suggesting that this particular column receives a stronger input or has a greater overlap with the input features compared to the others. Thus, the activation dynamics showcase how different neurons become active based on their varying responsiveness to the given input.

Characteristics of the Activation Curves

- **Order of Activation:** The orange line (Converter7) activates earlier than the blue (Converter3), followed by the other columns. This indicates that the orange column (Converter7) either receives a stronger synaptic input or has a more efficient connection, resulting in an earlier activation.
- **Steepness of the Curves:** The steep rise of each line indicates the strength and potency of synaptic input. A steeper curve represents a more direct or potent activation, possibly reflecting high synaptic weights or a higher number of inputs driving the neuron.
- **Plateau Phase:** After each curve rises steeply, it eventually reaches a plateau, representing a stable activation state. This plateau indicates that the neurons or columns have reached their maximum activation levels and are now actively representing the input. This stabilization phase is critical, as it demonstrates that the network has converged, meaning that the competition is over, and the winning neurons are consistently representing the input pattern.

Understanding the KWTa Mechanism

The kWTa mechanism ensures that only a specific number ('k') of columns become active, which allows the network to maintain sparsity in its representation of input. By analyzing the number of lines that remain active in the graph, we can infer the value of 'k' used in this competition. In the given plot, there are multiple columns active, indicating the system is enforcing a sparsity constraint, selecting only a subset of the neurons as winners. For instance, if we observe that two curves reach a plateau and remain active while others do not, this might indicate that columns were allowed to remain active, implying a sparse representation.

Activation Threshold and Feature Detection

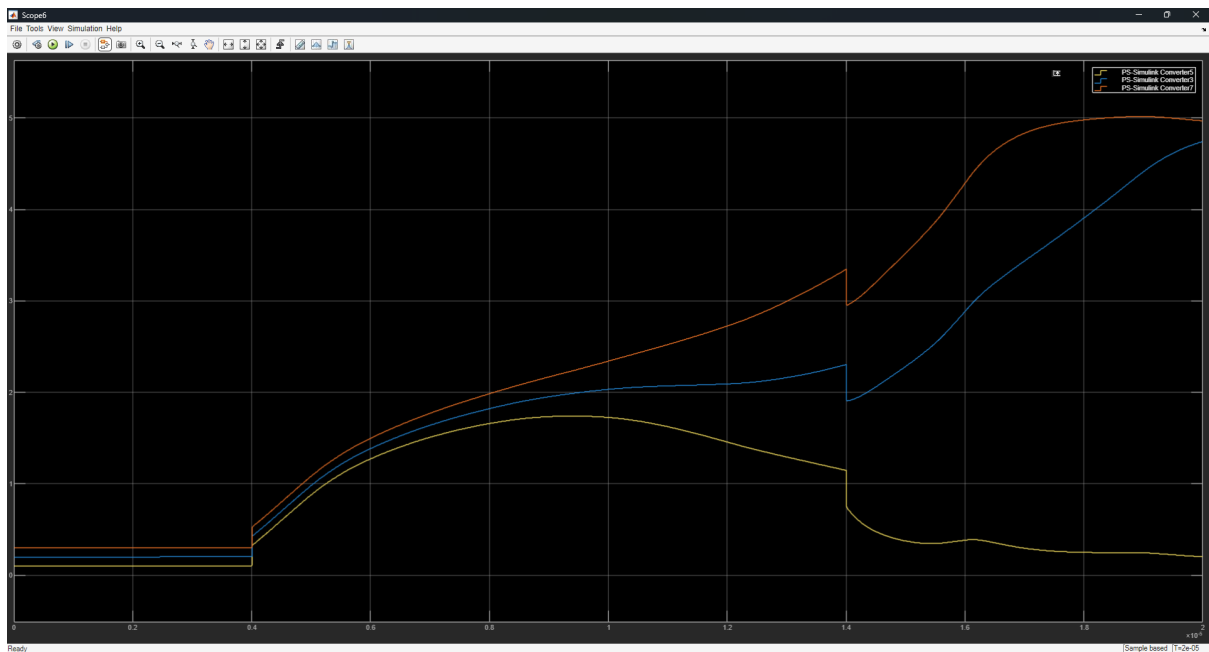
The activation threshold, observed between 0.8 to 1.0 (time units), is a key point in understanding when neurons start responding to the input. In this graph, each line's activation reflects how well the corresponding neuron overlaps with the features of the input. The differences in activation timing and curve steepness are essential indicators of how different features compete to be represented by the network. The orange line activates first, suggesting that it has the best match with the features of the current input, followed by the blue line.

Thus, the kWTa mechanism effectively allows the HTM network to identify and represent the most prominent features of the input while maintaining a sparse, efficient activation pattern. The plateau phase indicates stability in the winning columns, which contributes to a reliable representation of the input data.

Summary (Graph 1 (Scope 1))

In summary, the graph illustrates the competitive activation process of multiple neurons in an HTM network, showing how neurons cross an activation threshold to represent input features. The kWTA mechanism ensures that only a subset of neurons becomes active, preserving sparsity and enabling the network to focus on the most relevant features. The observed dynamics — including the activation threshold, order of activation, steepness, and stabilization — This shows how the network selectively activates neurons in response to incoming stimuli, which is used for effective feature representation and learning.

Graph 2 (Scope 6):



The graph (Scope 6) illustrates both the activation and adaptation behaviors of neurons over time. Here, the dynamics not only include the activation of columns but also the decay or adaptation of others, highlighting the temporal evolution of synaptic permanence.

Adaptation and Activation Dynamics (Scope 6)

- **Distinct Rise and Stabilization:** Some curves in the graph show a distinct rise and then stabilize, indicating that certain neurons have successfully become winners and maintain their activation levels over time. This behavior reflects how frequently active neurons strengthen their synaptic connections and maintain consistent representation of input features.
- **Decline of Some Curves:** Notably, some curves exhibit a decline after an initial rise. This suggests that certain neurons, which were initially active, begin to decay over time. This behavior lines up with the concept of synaptic permanence adjustment, where neurons that are not frequently stimulated gradually weaken and lose their activation. This decay is critical in ensuring that the network remains efficient and that only the most relevant neurons remain active.
- **Order and Overlap of Activation:** The order in which the curves rise and stabilize provides understanding into which columns have the highest overlap with the input features. In the graph, the orange and blue lines rise earlier, indicating that these columns are initially more relevant to the current input. However, as time progresses, the yellow line declines, suggesting that it was less effective in keeping a strong representation of the input.
- **Competitive Inhibition and Sparse Representation:** The KWTa dynamics observed in the graph reflect a biologically inspired competitive process, where only the most

relevant features are represented over time. This competitive inhibition mechanism ensures that only a subset of neurons is consistently active, achieving a sparse and efficient representation of the input. The decline of less active neurons contributes to the adaptive nature of the network, where synaptic connections are strengthened or weakened based on their relevance and frequency of activation.

Summary of Graph 2 (Scope 6)

In summary, the graph illustrates the competitive activation and adaptation processes of multiple neurons in an HTM network, this shows how neurons cross an activation threshold to represent input features and adapt over time. The kWTA mechanism ensures that only a subset of neurons becomes active, preserving sparsity and enabling the network to focus on the most relevant features. The observed dynamics including the activation threshold, order of activation, stabilization, and decline highlight how the network selectively activates and adapts neurons in response to incoming stimuli, which is used in effective feature representation, learning, and memory consolidation.

MATLAB Scripts

data_process.m:

This handles preprocessing of training and the testing data. It sorts labels and data, calculates the occurrences for each label, and normalizes samples by subtracting the mean image which aids convergence during training. Preprocessing acts as the initial phase, preparing data for input into the HTM spatial pooler and temporal memory modules.

get_mnist.m:

This script loads and processes the MNIST dataset of handwritten digits. MNIST is frequently utilized for evaluating image classification systems. Images undergo normalization and reshaping to be compatible with HTM processing, serving as the initial input for further learning.

new_sp.m (Spatial Pooler Module):

This script implements training and testing of the spatial pooler using memristor-based technology.

Key parameters and hyperparameters:

- **potential_radius:** Defines the area over which a column forms potential synapses determining the receptive field size.
- **column_dimension:** Specifies the dimensionality of HTM columns.
- **local_area_density:** Controls the sparsity of active columns for ensuring effective sparse distributed representation (SDR).
- **syn_dec** and **syn_inc:** Define rates for decrement and increment of synaptic permanence during learning.
- **inhibition_radius:** Determines the range within which columns compete for activation.

Training and Testing Process:

The initialization phase is setting random permanence values for synapses which determine the initial active connections. During training, each input sample is processed by the spatial pooler, adjusting synaptic permanence ("w_permanence") values based on predictive accuracy. **Overlap Calculation** determines the extent to which each column's synapses align with the current input, influencing which columns become active. The network then employs **Inhibition and Sparsity Enforcement** to ensure that only a sparse subset of columns is activated, achieving a sparse distributed representation.

Key Techniques for Improvement:

- **Local Area Density Adjustment:** Dynamically adjusting local_area_density balances learning efficiency with representation robustness.

- **Permanence Optimization:** More sophisticated optimization strategies, such as adaptive learning rates for synaptic increment (`syn_inc`) and decrement (`syn_dec`), could enhance permanence adjustments.
- **Memristor Modeling:** Leveraging memristor properties to emulate synaptic weights efficiently requires exploring different memristor behaviors, such as non-linearity or noise, and evaluating their effects on HTM learning. Utilizing memristor-based crossbars provides significant improvements in training speed and energy efficiency by enabling parallel training of multiple synapses.

Approaches for HTM Improvement

Enhanced SDR Generation: Improving sparsity constraints and potential synapse connections contributes to generating more efficient SDRs, thereby enhancing generalization. The integration of advanced SDR encoding techniques, which ensure semantically similar inputs produce highly overlapping SDRs, can significantly enhance the system's capacity to generalize across input types.

Advanced Temporal Memory Mechanisms: Integrating a more sophisticated temporal memory module, including multiple contexts or hierarchical layers, enhances predictive capacity by capturing long-term dependencies. Utilizing spin-neurons for these mechanisms can further reduce energy consumption and increase computational efficiency.

Learning Stability Mechanisms: Introducing mechanisms such as permanence boosting aids in synapse stabilization, and maintaining frequently used pathways.

Handling Memristor Noise: Given inherent variability in memristors, the application of noise filtering or averaging across multiple updates enhances network performance. Addressing challenges such as sneak currents and parasitic effects in memristor crossbars is crucial for improving the reliability of hardware implementations.

Simulink KWTa Block Analysis

The KWTa (k-Winners-Take-All) block likely represents a mechanism ensuring competitive activation. The KWTa structure ensures only the strongest neuronal responses are activated, enforcing sparse activation patterns characteristic of HTMs. The CMOS-based KWTa circuit design, with adjustable current ratios for controlling the number of winners, offers a highly efficient and biologically inspired method for competitive neuron selection. In the presented Simulink model, neurons and synapses are represented by individual blocks, with KWTa ensuring activation of only a small fraction of the columns.

Future Work and Demonstration of Improvements

To demonstrate the efficacy of the proposed approach, I am thinking of using several metrics and benchmarks can be employed:

Benchmarking: Comparisons should be drawn between the HTM-based system and conventional models such as convolutional neural networks (CNNs) or recurrent neural networks (RNNs) using the MNIST dataset.

Metric Analysis: Performance evaluation should include accuracy, anomaly detection rates, and prediction errors. Additionally, the effect of hyperparameters such as `local_area_density` and `inhibition_radius` on these metrics should be investigated.

Energy Efficiency Analysis: Given the biologically inspired nature of HTM, energy efficiency can be compared with traditional methods, showing the benefits of memristor-based, hardware-efficient implementations. The use of spin-neurons and resistive crossbars for implementing temporal memory further underscores the potential for drastic energy reductions compared to standard hardware.

Conclusion

Hierarchical Temporal Memory represents a biologically plausible, sparse approach to machine learning focused on temporal pattern and sequence learning. The MATLAB code shared effectively implements spatial pooling functionality, a fundamental aspect of HTM's method for learning representations. Future improvements in HTM can be achieved through refined sparsity mechanisms, adaptive synaptic updates, and leveraging memristor technology. The ongoing pursuit of biologically inspired machine learning, incorporating innovations like spin-neurons and memristor-based crossbars, could potentially yield systems capable of more robust and energy-efficient learning processes, advancing the capabilities of artificial intelligence.